HW-M4

Start Assignment

- Due Sunday by 11:59pm
- Points 20
- Submitting a file upload
- File Types py
- Available until Jun 4 at 11:59pm

Create a GUI that calculates your wealth for each of the N years after you start work (assume N = 50 in this assignment) given the following inputs into the GUI:

1. Mean Return (%) This is the average annual return of the investment above inflation
2. Std Dev Return (%) This is a measure of the annual volatility of the investment
3. Yearly Contribution ($)
4. No. of Years of Contribution
5. No. of Years to Retirement
6. Annual Spend in Retirement

There should be Quit and Calculate buttons at the bottom. When you press calculate, a function should be called which uses the specified values to run the analysis M times (assume M = 15 in this assignment), each time for N years. The GUI should display the average of those M times of your wealth at **retirement**. The retirement should be in a label, not in an entry field!

Your python code should also plot your wealth as a function of year for each of the M analyses. The plot must show how much money is in the account for all N years. (If the money runs out before N years for a particular analysis, the curve for that analysis should terminate at the x-axis at the year the money runs out. That is, you can't go below $0! And don't continue the line from where it goes to $0 to the end of the N years. The line ends when it hits the x axis.)

Following are some example formulas that capture the dynamics of the retirement system that your code is implementing. Your code must be based on these formulas. The wealth in year (i+1) can be calculated from the wealth at year i as follows:

1. From the start until contributions end: $W_{i+1}=W_i(1+(r/100)+noise[i]) + Y$
2. From end of contributions until retirement: $W_{i+1}=W_i(1+(r/100)+noise[i])$
3. From retirement to the end: $W_{i+1}=W_i(1+(r/100)+noise[i]) - S$

where

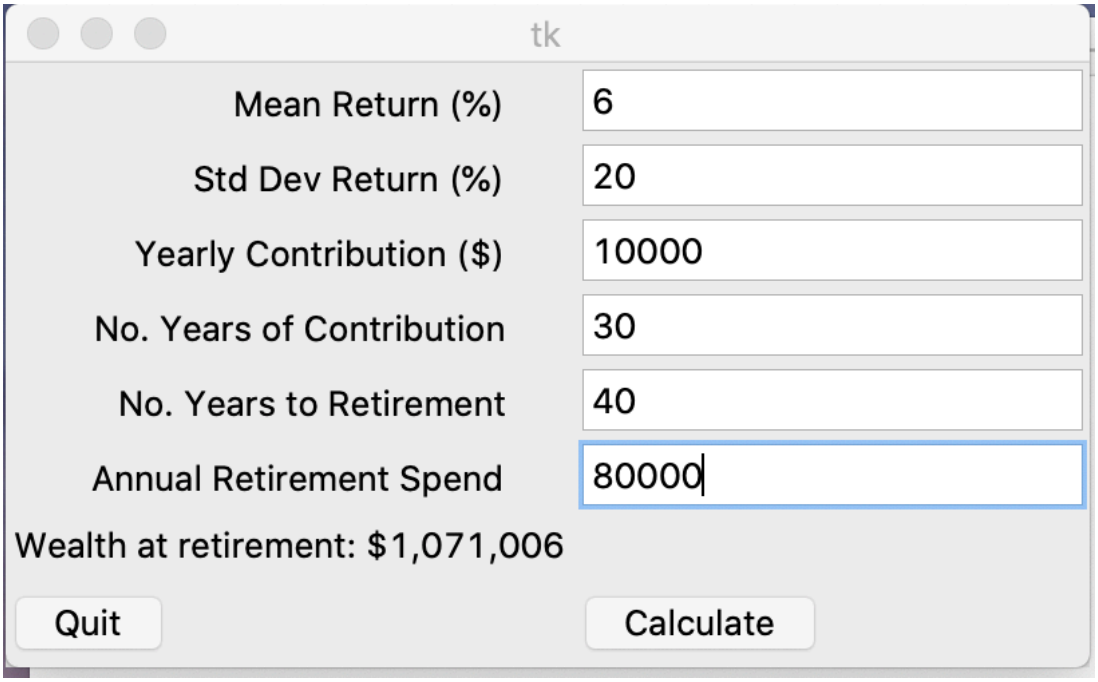
- r is the rate entered into the GUI,
- Y is the amount contributed each year (from the start until contributions stop),
- S is the amount spent each year in retirement (only following retirement), and

- `noise` is an array of N random values generated according to `noise = (sigma/100)*np.random.randn(MAX_YEARS)` where `sigma` is the standard deviation entered into the GUI, and `MAX_YEARS` is N which, in your script, should be a constant!

NOTES:

- Start with $0 in year 0
- Contributions must stop at, or prior to, retirement.
- Withdrawals start the year **after** retirement
- If years to retirement is the same as years of contributions, then the function of equation (2) above is skipped
- How to implement the above formulas is up to you.
  - If you are a beginner in programming, you are encouraged to implement them as they are presented above.
  - If you are advanced, you might be able to come up with efficient ways to implement these formulas.
  - You will not lose points if you implement them in any correct way even if inefficient.
- Add comments
- **You are encouraged to use tkinter or any other package**
- Turn in a file called hw4.py.

Here are examples of the GUI and the resulting graphs. Note that, due to randomness, you'll not be able to duplicate these exact numbers. Also, note that N and M in the following images might be different than the values you will use in your code according to the specifications above.



Some installations of Python have a strange problem where the update to the average wealth at retirement doesn't show up until after the plot is closed. There are two possible ways to solve this problem. One, or both, may work for you:

Try either importing matplotlib like this:

```
import matplotlib
matplotlib.use("TkAgg")
import matplotlib.pyplot as plt
```

Or try doing `plt.show()` like this:

```
plt.show(block=False)
```